Parking Agents' Context on the SSD: Memory-Aware Agent Serving on an Edge LLM Server

Kai-Cheng Liu
University of California San Diego
CSE 291A, Fall 2026 — Final report
Code and data: https://github.com/kcliu23/cse291A

***Abstract*—LLM agents alternate between model calls and tool calls, and every call resends a conversation that keeps growing. On a single edge server with a fixed memory budget, this forces a choice: admit few agents and leave the accelerator idle while they wait on tools, or admit many and let them overwrite each other's cached context. We characterize this trade-off on llama.cpp with a 3B model on an Apple M2 Mac and an AWS L4 GPU slice, find that context eviction is the dominant bottleneck, and build a client-side slot manager that gives each agent slot affinity and parks a waiting agent's KV cache on the local SSD when its slot is needed. Restoring a parked 1,000–3,500-token context takes 0.1–0.4 s, even when read from the SSD, versus 0.7–10 s to recompute it. On held-out workloads whose agents make long tool calls (5–30 s), parking lowers p95 task latency by 8–36% and raises throughput by up to 30% over the strongest baseline on both machines, on every held-out workload except the lightest, with no change in server memory; with short tool calls it is neutral. We also find that llama.cpp's similarity-based slot selection evicts agents that share a system prompt even when a free slot exists, and that a simple affinity rule fixes it.**

I. Introduction

LLM agents run a loop [1]: call the model, run a tool such as a search or a code interpreter, then call the model again with the whole conversation so far. Prompts grow at every step, and between steps the agent leaves the model idle. A serving system keeps each active conversation in its key–value (KV) cache so that it need not reprocess the history, but the cache is bounded by memory. Admitting few agents leaves the hardware idle during tool waits; admitting many overflows the cache, so returning agents find their context evicted and must recompute it. The dilemma is sharpest on edge servers, where one machine with a few gigabytes serves several agents and memory cannot be added.

This project asks how an edge server should admit and hold agents under a fixed memory budget. In Phase 1 we built a reproducible agent workload and harness, tuned a FIFO baseline with a fixed concurrency limit K, and showed that the bottleneck is context eviction once K exceeds the server's slots. In Phase 2 we built and evaluated a remedy that changes *where* agent context lives rather than only *when* agents run: waiting agents' context moves to the local SSD and comes back when they return. It pays off most when agents make long tool calls; otherwise it matches the best baseline and removes the penalty of admitting more agents than slots. Our contributions are:

- A characterization of agent serving on llama.cpp on two platforms whose prompt-processing speed differs 4× (Sec. III).
- A client-side slot manager that adds slot affinity and parks waiting agents' KV caches on the local SSD through llama-server's slot save/restore interface, with no server changes (Sec. IV).
- An evaluation on held-out workloads on both platforms, showing when parking helps (long tool calls) and when it does not (short tool calls), with its overhead (Sec. V).
- A measured flaw in llama.cpp's slot selection for agents that share a system prompt, and negative results for three further ideas (Sec. V-D, V-E).

II. Background and Related Work

KV cache and slots. A transformer server stores the keys and values of processed tokens; for Llama-3.2-3B this costs 112 KiB per token in f16. llama.cpp [2] splits a fixed KV allocation evenly among *slots*, each serving one request at a time. A slot keeps its last prompt, so a request whose prefix is still in some slot reuses it (*prompt caching*); otherwise the history is processed again. vLLM [3] instead pages one shared pool, and SGLang [4] shares prefixes through a radix tree.

Context beyond GPU memory. CachedAttention [5] keeps multi-turn conversations' KV caches in host memory and disk with layer-wise prefetching, and KVSwap [6] offloads a long context to disk on mobile devices with unified memory.

Serving agents. InferCept [7] decides per request whether to preserve, discard or swap context during a tool call; Continuum [8] pins it for a time-to-live; KVFlow [9] evicts and prefetches by each agent's next step in a workflow; Autellix [10] schedules calls by program progress. These systems target datacenter GPUs and paged memory. We study one edge machine with fixed slots and a hard memory

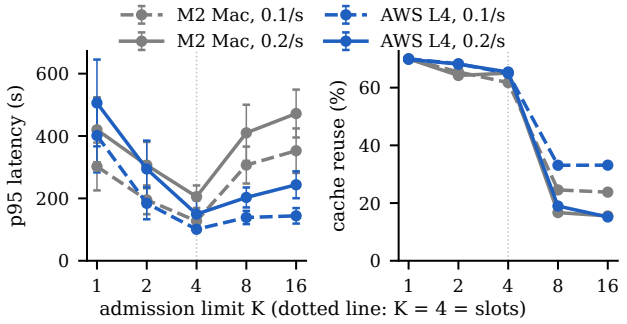


Fig. 1. Phase 1: FIFO with fixed K (short tool calls, mean of 3 seeds). Latency is lowest at $K = 4$, the slot count; above it, agents evict each other's context and prompt-cache reuse collapses.

budget, use its local SSD as the parking tier, and measure the trade-off on two platforms with very different recomputation costs.

III. Phase 1: Characterizing the Bottleneck

Setup. The primary platform is an Apple M2 Mac mini (24 GB unified memory, Metal); the second is an AWS g6f.2xlarge with an NVIDIA L4-6Q (a quarter of an L4, 6 GiB, CUDA) and a local NVMe instance-store SSD. Both run llama.cpp b11803 with Llama-3.2-3B-Instruct Q4_K_M (SHA-256 checked) and 4 slots of 4,096 tokens, allocating 4.1 GB (weights 2.2, KV 1.8, compute 0.2) of a 6 GB budget. llama.cpp's host-RAM prompt cache is disabled so memory equals this allocation. Prompt processing runs at 365–427 tokens/s on the Mac and 1,463–1,651 tokens/s on AWS; single-request generation at 37.5 vs. 23.3 tokens/s.

Workload. A seeded generator produces four task classes sharing a 200-token system prompt: *chat* (30%, 1 step), *RAG* (20%, 1 step, 2–3k-token context), *short agents* (30%, 3 steps) and *long agents* (20%, 6 steps, 0.8–1.5k-token context). Agent steps resend the full history plus a 100–400-token tool result, so contexts grow 200–600 tokens per step. Prompts are filler text and every step generates a fixed number of tokens (64–160, end-of-sequence ignored), so no policy can win by shortening answers; we measure time and memory, not answer quality. Tasks arrive as a Poisson process; each run has 30 tasks and starts a fresh server. Seeds 0–2 were used for development and seeds 1000–1004 only for the final evaluation.

Metrics. *Task latency* runs from a task's arrival to its last token, including queueing and tool waits; we report its 95th percentile (p95). *Throughput* is completed tasks divided by the time from the first arrival to the last completion. *Prompt-cache reuse* is the share of prompt tokens served from a slot's cache.

Findings. Fig. 1 summarizes 90 runs. On both machines the best fixed K equals the slot count. Raising K above it worsens latency, throughput and reuse: from $K = 4$ to $K = 8$ at 0.2 tasks/s, prompt-cache reuse falls from 65% to 17–19% and p95 rises 2.0× on the Mac but 1.4× on AWS (Table I). A returning agent usually finds its slot reassigned and reprocesses its whole history; each request spends 2.4 s more on prompt processing on the Mac but only 0.6 s on AWS, and these long recomputations also slow the generation of requests batched with them. A causal test confirmed the cause (Fig. 2): keeping evicted prompts in llama.cpp's host-RAM cache at $K = 8$ restored 70% reuse and the best fixed K's latency, but needed 11.8 GB; a 1 GB cache within budget recovered about 60% of the gap. An 8-bit KV cache fits 8 slots in about the KV memory of 4 (1.9 vs. 1.8 GB), and its outputs matched f16 exactly on 50 scored prompts. The penalty for exceeding the slots is the reuse lost times the recomputation cost, which motivated Phase 2: keep agents' context across tool calls without spending the memory budget on it.

TABLE I
Phase 1: going from $K = 4$ to $K = 8$ at 0.2 tasks/s (mean of 3 seeds)

	M2 Mac	AWS L4-6Q
Prompt-cache reuse	65% → 17%	65% → 19%
Median prompt time / request	1.17 → 3.55 s	0.37 → 0.96 s
Generation / request	12.3 → 8.9 tok/s	12.4 → 10.8 tok/s
p95 task latency	205 → 410 s	149 → 203 s

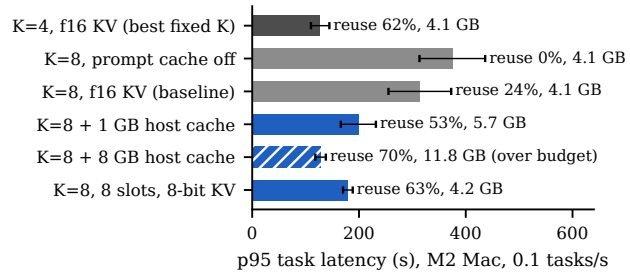


Fig. 2. Phase 1 causal test: $K = 8$ on the M2 Mac at 0.1 tasks/s (mean ±1 std of 3 seeds; memory is weights plus measured footprint). Retaining evicted context recovers latency, but llama.cpp's host-RAM cache needs memory beyond the budget to do so (hatched).

IV. Design

Overview. Fig. 4 shows the design. Admission stays FIFO with a limit K, which may now exceed the slot count. A slot manager in the client then decides, for every model call, which slot it runs in and what happens to the context it displaces. It tracks which agent's context is resident in each slot, which slots are busy, and which agents are waiting on a tool. It sends each request with an explicit slot id, so llama-server never chooses a slot on its own.

Slot choice. An agent whose context is resident in an idle slot always gets that slot (*affinity*). Otherwise free slots go first to returning agents, which are partway through a task,

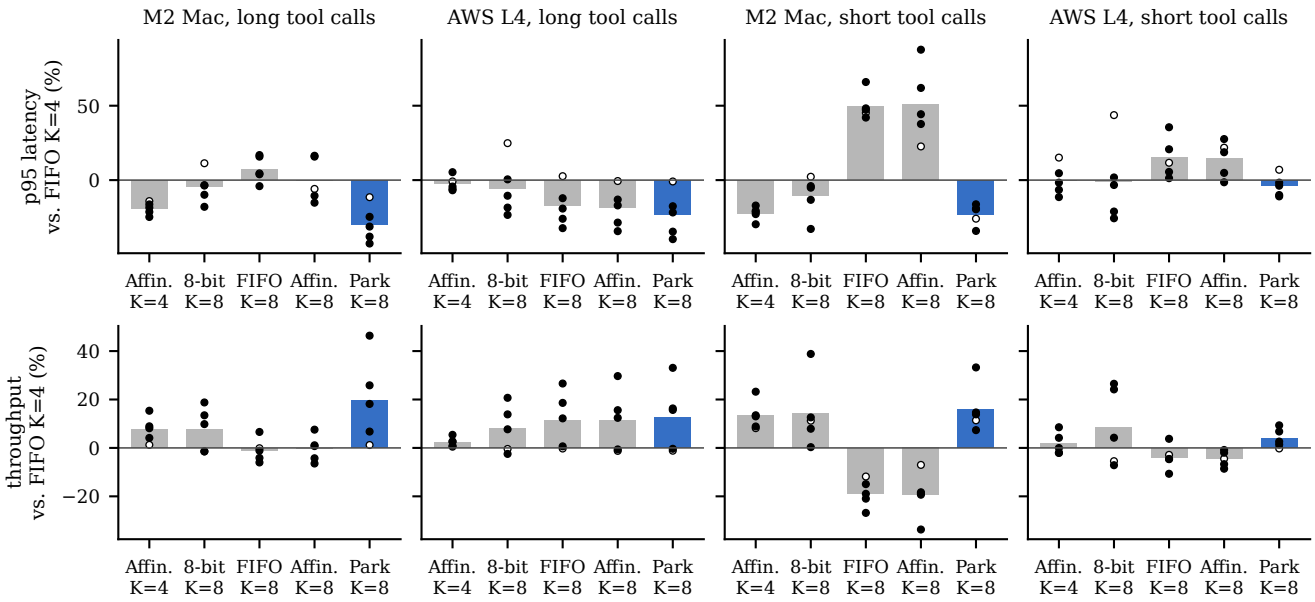


Fig. 3. Held-out evaluation: p95 task latency (top) and throughput (bottom) relative to FIFO at $K = 4$ with server slot choice, on the same workload seed. Bars: mean of seeds 1000–1004; dots: individual seeds (hollow: seed 1000, the lightest). Parking at $K = 8$ gives the largest gains with long tool calls on both machines; with short tool calls it matches affinity at $K = 4$, and its gain over FIFO $K = 4$ there comes from affinity.

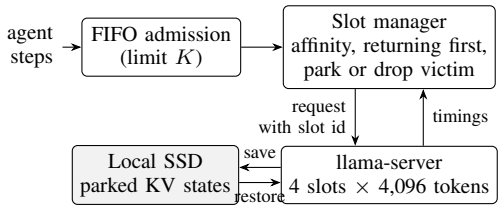


Fig. 4. The slot manager sits in the client between admission and llama-server. It chooses the slot of every request and moves waiting agents' context between slots and the SSD through llama-server's slot save/restore endpoints.

and then to new tasks in arrival order. An agent takes an empty slot if one exists; only when every idle slot holds the context of an agent waiting on a tool does it evict one, choosing the least recently used; if all slots are busy it waits. Without parking, this policy is our *affinity* baseline: evicted context is simply dropped.

Parking. Before evicting, the manager saves the victim slot's KV state to a file on the local SSD (`POST /slots/{id}?action=save`); when the victim returns, it restores the file into a free slot (`action=restore`) and resumes with its whole history reused. Saving happens only when a slot is actually needed, so agents that keep their slot pay nothing; each agent has at most one parked copy, deleted when it is restored or the task ends. A parked context takes about 112 KiB per token on disk, the same as in memory, so the SSD rather than the 6 GB budget holds waiting agents.

Implementation. The manager is under 300 lines of asyncio Python in our harness and needs no change to llama.cpp; the server only needs `-slot-save-path`. It logs every save, restore and drop with its duration and size. On AWS the parked files go to the instance's local NVMe SSD, since the root volume is a small network disk. A mock llama-server with simulated save/restore let us test the pipeline without a model.

V. Evaluation

Methodology. We evaluate on held-out seeds 1000–1004 on both machines and two workloads: the Phase 1 mix at 0.2 tasks/s (*short tool calls*, 0.5–4 s) and the same mix with tool calls of 5–15 s for short agents and 10–30 s for long agents at 0.1 tasks/s (*long tool calls*, as for web search or code execution). The two arrival rates were chosen so that both workloads overload FIFO at $K = 4$. We compare FIFO at $K = 4$ with server slot choice (the Phase 1 baseline); affinity at $K = 4$, our slot choice without parking, which is the strongest $K = 4$ baseline; FIFO and affinity at $K = 8$, which admit as many agents as parking does but drop evicted context; the best configuration-only baseline from Phase 1, 8 slots with an 8-bit KV cache; and parking at $K = 8$ and 16. Each configuration runs once per seed. Seeds differ strongly in load because the task mix is random: seeds 1000 and 1003 contain 0 and 1 long agents, seeds 1001, 1002 and 1004 contain 7, 8 and 11, and seed 1000 also has the least total work, so every configuration finishes it with p95 under 100 s. We therefore compare configurations on the same seed and show every seed.

TABLE II
P95 LATENCY (S) / THROUGHPUT (TASKS/S), MEAN OF HELD-OUT SEEDS 1000–1004

	Long tool calls		Short tool calls	
	Mac	AWS	Mac	AWS
FIFO $K = 4$	217 / .061	184 / .064	180 / .099	118 / .119
Affinity $K = 4$	176 / .065	175 / .065	138 / .110	114 / .121
8-bit, 8 slots, $K = 8$	197 / .065	161 / .068	152 / .110	**104 / .124**
Affinity $K = 8$	226 / .061	139 / .069	281 / .082	136 / .114
Parking, $K = 8$	**143 / .071**	**128 / .070**	**137 / .112**	112 / .123

TABLE III
PARKING OVERHEAD (HELD-OUT RUNS WITH PARKING, $K = 8$ AND 16)

	M2 Mac	AWS L4
Save, median / p95 (ms)	43–60 / 102–137	40 / 76–77
Restore, median / p95 (ms)	45–80 / 193–217	34 / 65–66
Restore + resume, 1k / 3.5k (ms)*	190 / 340	105 / 145
read from SSD*	265 / 391	138 / 293
Recompute, 1k / 3.5k (ms)*	2,600 / 9,700–10,000	690 / 2,330
Save + restore share of run time	$\leq$2.4%	$\leq$1.7%
Peak parked (GB)	0.8–1.1	0.6–1.1
Server memory vs. FIFO $K = 4$	+<20 MB	+$\leq$2 MB GPU

*Controlled probe on an otherwise idle server.

A. Main results

Fig. 3 shows per-seed results; Table II lists absolute values.

Long tool calls. Parking has the lowest p95 and highest throughput on both machines. Against the strongest baseline, affinity at $K = 4$, it lowers p95 on seeds 1001–1004 by 8–27% on the Mac and 18–36% on AWS, and raises throughput by 3–27% and up to 30%; on seed 1000 the two are within 3%. Against the Phase 1 baseline the gains on seeds 1001–1004 are 18–43% in p95 and up to 46% in throughput, and against the 8-bit configuration 8–31% in p95. With $K = 4$, slots sit idle while agents wait on 5–30 s tools; admitting $K = 8$ fills those gaps, and parking keeps the extra admission from destroying context. The matched comparison at $K = 8$ isolates parking: on the Mac, affinity at $K = 8$ without parking is no better than FIFO at $K = 4$, and parking lowers its p95 by 19–46% on seeds 1001–1004; on AWS, where recomputation is 4× cheaper, dropping context costs less and parking adds 1–10%. $K = 16$ performs like $K = 8$.

Short tool calls. Parking is neutral against affinity at $K = 4$: within $\pm$8% on every seed and machine. Tool waits are too short for $K = 4$ to leave slots idle, so there is little for extra admission to use; at the same $K = 8$, however, parking is 39–65% better than dropping context on the Mac and 9–25% on AWS, so it removes the penalty of over-admission. The gain over the Phase 1 baseline here (16–34% p95 on the Mac) comes from affinity, not parking.

Who benefits. Short tasks gain most because they no longer queue behind agents that hold slots while waiting on tools: with long tool calls, chat p95 (mean of the five seeds) falls from 110 to 25 s on the Mac and from 86 to 14 s on AWS (FIFO $K = 4$ vs. parking). Long agents also improve, from 329 to 209 s and 292 to 193 s.

B. Overhead

Parking cost is small on both machines (Table III). In the runs, a save or restore took a median of 34–80 ms. In a controlled probe, restoring a 1,000–3,500-token context and resuming took 0.19–0.34 s on the Mac and 0.10–0.15 s on AWS, against 2.6–10.0 s and 0.7–2.3 s to recompute it: 14–28× and 7–16× faster. All saves and restores together occupied at most 2.4% of a run's time, which bounds their slowdown of other agents even if they stalled the server completely. Server memory is unchanged: the Mac's footprint rose by under 20 MB and AWS GPU memory by at most 2 MB, while 0.6–1.1 GB of context was parked at peak.

The file cache. Writing 200 MB in 40–60 ms is faster than the SSD itself, so the operating system's file cache absorbs the writes, and recent files may be read back from RAM. That memory belongs to the OS and is reclaimed under pressure, unlike llama.cpp's host-RAM prompt cache, which we counted against the budget in Phase 1. To bound the effect, we repeated the probe after dropping the file cache before each restore, so every restore read from the SSD. A 3,500-token restore then took 167 ms on the Mac and 208 ms on AWS (2.3 and 1.8 GB/s) instead of 136 and 62 ms, and restoring with resuming stayed 10–26× faster than recomputing on the Mac and 5–8× on AWS (Table III). Parking therefore does not depend on the file cache.

C. Slot affinity: a flaw in llama.cpp's slot choice

Even at $K = 4$, where every agent could keep its own slot, FIFO with server slot choice lost a returning agent's context in 79–85 of 225 returning steps with long tool calls and 18–27 of 225 with short ones. In every case a free slot existed. llama-server picks the idle slot whose cached prompt shares the longest prefix with the new request, if the shared fraction exceeds 10%, and otherwise the least recently used. Because all agents share a 200-token system prompt, a new short request matches every slot, including one holding a waiting agent's 2,000-token conversation, and overwrites it. Our manager's affinity rule (prefer empty slots, never take a waiting agent's slot while another is free) removed all of these evictions. Alone, it lowered p95 by 14–30% on every Mac seed; on AWS, where recomputing is cheap, the effect was between -11% and $+15$%. This is a scheduling fix, distinct from parking, and we report the two separately.

D. What did not work

Cost-aware parking. We built a variant that drops contexts too short to be worth saving and evicts the agent expected to return last, after KVFlow [9]. On development seeds it performed the same as plain parking: no evicted

context was short enough to drop, and choosing victims by expected return gave the same latency as LRU. Since restoring costs about 0.2 s per task, there is little for a cost model to decide on our hardware.

Restoring early. Prefetching a context before the tool call ends, as CachedAttention and KVFlow do, could save at most the restore time. On development runs that was 0.15 s of an 81 s mean task on the Mac, so we did not build it. The larger remaining delay is waiting for one of the four slots (7–15 s per task), not data movement.

Parking plus 8-bit KV. Combining parking with 8 slots of 8-bit KV cut slot waiting from 27 to 7 s on development seeds but raised p95 (Mac 210 to 248 s): with 8 requests generating at once, each decodes more slowly and long agents finish later. Throughput was unchanged; the accelerator's compute, not memory, became the limit. More slots shift latency from short to long tasks rather than reducing it.

VI. Discussion

When should an edge server park? Parking pays off when three conditions hold together: more agents are waiting on tools than there are slots, their contexts are long, and the tool calls are long enough that slots would otherwise idle. The first two make eviction frequent and expensive; the third makes extra admission useful. Our short-tool-call workload meets the first two but not the third, so parking only matched the best fixed K; seed 1000, the lightest, rarely needs a slot it does not have and shows no gain. Because a save or restore costs tens of milliseconds while recomputation costs seconds, the decision to park, once a slot is needed, is rarely close on our hardware; the real decision is how many agents to admit, and parking makes admitting more of them safe.

Hardware. The two platforms differ $4\times$ in prompt-processing speed, yet parking helped on both. The difference shows up in the baselines instead: without parking, admitting more agents hurts more on the Mac, where lost context is expensive to rebuild: with short tool calls, affinity at $K = 8$ raises p95 over FIFO $K = 4$ by 23–88% on the Mac but by at most 28% on AWS, and with long tool calls on AWS it even helps. A larger model would raise both the recomputation cost and the size of each parked file (KV bytes per token grow with layers and heads), so parking should matter more, not less, as long as the SSD can absorb the traffic; at our scale a run wrote 3–4 GB on average and at most 7.7 GB, a modest load for an SSD but one that grows with model size and agent count.

Toward the server. Our manager runs in the client and needs only the existing slot save/restore endpoints, which kept the server untouched but costs a file write per eviction. Inside llama.cpp, the same policy could copy KV blocks to host memory or the SSD asynchronously and fix the slot-selection flaw for all clients; the affinity rule alone is a few lines in the slot chooser.

VII. Experience

What went wrong. A sweep that we interrupted left its llama-server running on port 8080; the next sweep's servers failed to bind, while the stray one answered every request, so 22 runs silently measured the wrong configuration. We detected it from a slot count that did not match the configuration, discarded and reran the runs, and made the harness refuse a busy port and verify that the answering server is its own. A pilot's memory grew to 7 GB, which we traced to llama.cpp's default 8 GB host-RAM prompt cache; we disabled it for budgeted runs and studied it separately. Back-to-back streamed requests failed with "server disconnected" because the client reused connections the server had already closed; fresh connections per request fixed it. Our first cache-hit metric counted the whole slot, including the previous answer, rather than reused prompt tokens. Operationally, cloud-synced folders on the Mac reverted edited files several times, and the AWS root disk had 4 GB free, too little for parked contexts, until we used the instance's NVMe store.

What we learned. Two predictions failed. We expected parking to lose on AWS, where recomputation is cheap and we assumed a network disk; on the local NVMe, saves and restores were even faster than on the Mac. We expected parking to beat the best fixed K on the original workload; it only matched it, which led us to the long-tool-call experiment where it does win. Measuring the ceiling of an idea before building it (restore-early) saved a week. Large load differences between seeds made means with standard deviations misleading, so comparing configurations on the same seed was essential.

What worked well. A mock server for testing the pipeline without a model; a fresh server, fixed output lengths and seeded workloads for every run; resumable sweeps with completion markers, which survived interruptions and a $3/day cloud budget; logging every request, task, and context movement so that new questions (such as the slot-affinity flaw) could be answered from existing runs; and held-out seeds for the final evaluation.

VIII. Limitations and Conclusion

The workload is synthetic: filler text, fixed output lengths and chosen tool delays, not traces of real agents, and the benefit depends on the share of long agents. We evaluate one 3B model on llama.cpp's fixed slots; paged servers degrade differently. Each configuration ran once on each of five held-out seeds, so we report per-seed differences rather than confidence intervals. Parked files pass through the OS file cache (Sec. V-B shows parking remains fast without it), and the Mac's memory budget is enforced by configuration rather than hardware.

On an edge server, agents that wait on tools should not hold their slot's memory, and should not lose their context either. Parking it on the local SSD is cheap, a few hundred milliseconds at most per move, and turns idle slots into

useful work when tool calls are long, lowering p95 latency by up to 36% and raising throughput by up to 30% over the strongest baseline without adding server memory. When tool calls are short, a simple slot-affinity rule is enough.

REFERENCES

[1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations (ICLR)*, 2023.

[2] ggml-org, "llama.cpp: LLM inference in C/C++," https://github.com/ggml-org/llama.cpp, build b11803 (commit a37cc461), 2026.

[3] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with PagedAttention," in *Proc. 29th ACM Symposium on Operating Systems Principles (SOSP)*, 2023.

[4] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu *et al.*, "SGLang: Efficient execution of structured language model programs," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[5] B. Gao, Z. He, P. Sharma, Q. Kang, D. Jevdjic, J. Deng, X. Yang, Z. Yu, and P. Zuo, "Cost-efficient large language model serving for multi-turn conversations with CachedAttention," in *Proc. USENIX Annual Technical Conference (ATC)*, 2024.

[6] H. Zhang, C. Xia, and Z. Wang, "KVSwap: Disk-aware KV cache offloading for long-context on-device inference," arXiv:2511.11907, 2025.

[7] R. Abhyankar, Z. He, V. Srivatsa, H. Zhang, and Y. Zhang, "InferCept: Efficient intercept support for augmented large language model inference," in *Proc. 41st International Conference on Machine Learning (ICML)*, ser. PMLR, vol. 235, 2024, pp. 81–95.

[8] H. Li, R. He, Q. Mang, Q. Zhang, H. Mao, X. Chen, H. Zhou, H. Zhang, A. Cheung, J. Gonzalez, and I. Stoica, "Continuum: Efficient and robust multi-turn LLM agent scheduling with KV cache time-to-live," arXiv:2511.02230, 2025.

[9] Z. Pan, A. Patel, Z. Hu, Y. Shen, Y. Guan, W.-L. Li, L. Qin, Y. Wang, and Y. Ding, "KVFlow: Efficient prefix caching for accelerating LLM-based multi-agent workflows," arXiv:2507.07400, 2025.

[10] M. Luo, X. Shi, C. Cai, T. Zhang, J. Wong, Y. Wang, C. Wang, Y. Huang, Z. Chen, J. E. Gonzalez, and I. Stoica, "Autellix: An efficient serving engine for LLM agents as general programs," arXiv:2502.13965, 2025.